

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN CIENCIAS DE LA INGENIERÍA
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1
CATEDRÁTICO: ING. JOSE MOISES GRANADOS GUEVARA



PABLO DANIEL ALVARADO RODRIGUEZ

CARNÉ: 202130534

QUETZALTENANGO, MAYO 2026

Introducción

El siguiente documento ofrece al desarrollador detalles importantes sobre la arquitectura y el método de construcción de un sistema de creación de páginas web utilizando la herramienta de Jison, esto por el lado del Frontend.

Se espera que esta documentación sea suficiente para que el desarrollador/programador entienda la base de cómo funciona el sistema.

Índice

Tecnologías Utilizadas.....	4
Front-end.....	4
Back-end.....	4
Hardware.....	5
Software.....	5
Análisis de Gramática para el Analizador Léxico.....	5
Análisis de Gramática para el Analizador Sintáctico.....	7
Generación de Gramáticas.....	14
Calcular Primeros.....	14
Calcular Siguietes.....	16
Diagrama de Clases.....	18
Diagrama de clases del Front-end.....	18
Diagrama de clases del Back-end.....	18

Tecnologías Utilizadas

Front-end

- **Svelte:** Se utilizó este Framework para la realización de la app web ya que permite la inserción de código en tiempo real sin la necesidad de realizar un re-deploy a la app. Para este proyecto se utilizó la versión 5.55.1.
- **HTML:** Este es un lenguaje de marcado que permite crear la estructura de los sitios web, se utilizó la versión 5 de este lenguaje.
- **JavaScript:** Lenguaje de programación que permite desarrollar todas las acciones e interacciones de la app.
- **SCSS:** Se utilizó para darle estilos a la app y que fuera más amigable y atractiva para el usuario.
- **Node.Js:** Se utilizó esta tecnología para el backend interno de la aplicación. Permite la interacción entre la app y diferentes programas instalados en el sistema como Jison y otras librerías que necesitan nativamente el framework. Se utilizó la versión 22.20.0 de esta tecnología.
- **Jison:** Este es un LALR1 que nos permite la generación de gramáticas y también puede ser utilizado para la generación de analizadores sintácticos. En el caso de nuestro proyecto nos ayuda a generar analizadores descendentes LL1. Para este Proyecto se utilizó la versión 0.4.18.

Back-end

- **JavaScript:** Se utilizó este lenguaje debido a su versatilidad en la web, permitiendo compatibilidad entre Jison y Node.js.
-

Hardware

Las especificaciones del hardware utilizado son las siguientes

- **Sistema Operativo:** Se utilizó Ultramarín-Linux que es un sistema con base Fedora. La versión de la base Fedora fue Fedora 41.
- **Kernel:** Se utilizó el kernel 6.17.10-100 en el sistema operativo.
- **RAM:** Para la realización de este proyecto se utilizó una máquina con 16GB de memoria RAM debido a que se requería tener dos aplicaciones encendidas al mismo tiempo.
- **Almacenamiento:** La máquina utilizada cuenta con 240GB de almacenamiento utilizando un Disco de Estado Sólido (SSD).

Software

Para la realización de esta aplicación se utilizaron diferentes programas de software, los cuales por comodidad se recomienda.

- **Visual Studio Code (VSCode):** Utilizado para el desarrollo de la parte frontend.
- **Netbeans:** Utilizado para el desarrollo de la parte del backend.

Análisis de Gramática de Estilos para el Analizador Léxico

Para el analizador Léxico utilizamos las bondades que la librería Jison nos ofrece y por tanto a continuación se mostraran las expresiones regulares que se utilizaron para el lexer implementado.

```
%{  
  var erroresLexicos = [];  
  var erroresSintacticos = [];  
  parser.erroresLexicos = erroresLexicos;  
  parser.erroresSintacticos = erroresSintacticos;
```

```

function registrarErrorLexico(lexema, linea, columna){
    erroresLexicos.push({
        tipo: 'lexico',
        lexema: lexema,
        linea: linea,
        columna: columna,
        mensaje: 'Token no reconocido: ' + lexema
    });
}

function registrarErrorSintactico(mensaje, lexema, linea, columna){
    erroresSintacticos.push({
        tipo: 'sintactico',
        lexema: lexema,
        linea: linea,
        columna: columna,
        mensaje: mensaje
    });
}

function registrarErrorSintacticoActual(mensaje){
    var linea = (typeof yylloc !== 'undefined' && yylloc) ?
(yyllloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
    var columna = (typeof yylloc !== 'undefined' && yylloc) ?
(yyllloc.first_column || 0) : 0;
    var lexema = (typeof yytext !== 'undefined') ? yytext : '';
    registrarErrorSintactico(mensaje, lexema, linea, columna);
}

parser.parseError = function(str, hash) {
    registrarErrorSintactico(str, hash.text || hash.token, hash.line +
1, (hash.loc ? hash.loc.first_column : 0));
    if (!hash.recoverable) {
        throw new Error(str);
    }
};

%}

%lex
%options ranges yylineno
%%

```

```
\s+                /* ignorar espacios y saltos */
[\u200B\u200C\u200D\uFEFF\u00A0]+ /* ignorar invisibles y nbsp */

\\/\\*[\\s\\S]*?\\*\\/ /* ignorar comentarios de bloque */

/* Palabras reservadas */
"height"           return 'ALTO';
"width"            return 'ANCHO';
"min"              return 'MINIMO';
"max"              return 'MAXIMO';
"background"       return 'FONDO';
"color"            return 'COLOR';
"text"             return 'TEXTO';
"align"            return 'ALINEACION';
"size"             return 'SIZE';
"font"             return 'FUENTE';
"padding"          return 'PADDING';
"margin"           return 'MARGIN';
"left"             return 'LEFT';
"right"            return 'RIGHT';
"top"              return 'TOP';
"bottom"           return 'BOTTOM';
"border"           return 'BORDE';
"radius"           return 'RADIO';
"style"            return 'STYLE';
"extends"          return 'EXTIENDE';
"@for"             return 'PARA';
"from"             return 'DESDE';
"through"          return 'HASTA';
"to"              return 'HASTA_EXCLUYE';

/* Valores */
"CENTER"           return 'CENTER';
"RIGHT"            return 'RIGHT_VALUE';
"LEFT"             return 'LEFT_VALUE';
"HELVETICA"        return 'HELVETICA';
"SANS"             return 'SANS';
"SERIF"            return 'SERIF';
"MONO"             return 'MONO';
"CURSIVE"          return 'CURSIVE';
"DOTTED"           return 'DOTTED';
"LINE"             return 'LINE';
"DOUBLE"           return 'DOUBLE';
```

```

"SOLID"                return 'SOLID';
"rgb"                   return 'RGB';
"blue"                  return 'COLOR_BLUE';
"white"                 return 'COLOR_WHITE';
"red"                   return 'COLOR_RED';
"green"                 return 'COLOR_GREEN';
"violet"                return 'COLOR_VIOLET';
"gray"                  return 'COLOR_GRAY';
"black"                 return 'COLOR_BLACK';
"lightgray"             return 'COLOR_LIGHTGRAY';

/* Literales */
"#[0-9a-fA-F]{3}([0-9a-fA-F]{3})?" return 'COLOR_HEX';
[0-9]+("."[0-9]+)?"%" return 'PORCENTAJE';
[0-9]+("."[0-9]+ return 'DECIMAL';
[0-9]+ return 'ENTERO';
"$"[a-zA-Z\u00C0-\u017F_][a-zA-Z0-9_\u00C0-\u017F]*
return 'VARIABLE';
[a-zA-Z\u00C0-\u017F_][a-zA-Z0-9_\u00C0-\u017F]* return
'IDENTIFICADOR';
"#" /* comentarios */

/* Simbolos */
"{" return 'LLAVE_ABRE';
"}" return 'LLAVE_CIERRA';
"_" return 'GUION';
";" return 'PUNTO_COMA';
"==" return 'IGUALDAD';
"=" return 'ASIGNAR';
"@" return 'DECORADOR';
"*" return 'MULTIPLICADOR';
"/" return 'SLASH';
"==" return 'IGUALDAD';
"+" return 'SUMA';
"%" return 'MODULO';
"(" return 'PAREN_ABRE';
")" return 'PAREN_CIERRA';
"," return 'COMA';

<<EOF>> return 'EOF';

. {

```



```

var linea = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
var columna = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_column || 0) : 0;
resgistrarErrorLexico(yytext, linea, columna);
}

/lex

```

Análisis de Gramática de Estilos para el Analizador Sintáctico

Nuevamente Jison nos permite utilizarlo para generar nuestro analizador sintáctico permitiendo la declaración de terminales y no terminales, asimismo permite la estructuración de reglas de producción que nuestro lenguaje requiere.

```

/* Declaraciones de precedencia y asociatividad */
%left SUMA GUION
%left MULTIPLICADOR SLASH MODULO

%%

style_list
: style_definitions EOF
  { return $1; }
;

style_definitions
: style_definition
  { $$ = [$1]; }
| style_definitions style_definition
  { $1.push($2); $$ = $1; }
;

style_definition
: IDENTIFICADOR LLAVE_ABRE style_body LLAVE_CIERRA
  { $$ = { tipo: 'estilo', nombre: $1, propiedades: $3 }; }
| IDENTIFICADOR GUION VARIABLE LLAVE_ABRE style_body LLAVE_CIERRA

```

```

    { $$ = { tipo: 'estilo', nombre: $1 + '-' + $3, propiedades: $4 }; }
  | IDENTIFICADOR VARIABLE LLAVE_ABRE style_body LLAVE_CIERRA
    { $$ = { tipo: 'estilo', nombre: $1 + $2, propiedades: $3 }; }
  | IDENTIFICADOR EXTIENDE IDENTIFICADOR LLAVE_ABRE style_body
  LLAVE_CIERRA
    { $$ = { tipo: 'estilo', nombre: $1, heredaDe: $3, propiedades: $5
  }; }
  | IDENTIFICADOR EXTIENDE IDENTIFICADOR GUION VARIABLE LLAVE_ABRE
  style_body LLAVE_CIERRA
    { $$ = { tipo: 'estilo', nombre: $1, heredaDe: $3 + '-' + $4,
  propiedades: $6 }; }
  | IDENTIFICADOR EXTIENDE IDENTIFICADOR VARIABLE LLAVE_ABRE style_body
  LLAVE_CIERRA
    { $$ = { tipo: 'estilo', nombre: $1, heredaDe: $3 + $4, propiedades:
  $5 }; }
  | PARA variable DESDE expr_numerica_for HASTA expr_numerica_for
  LLAVE_ABRE style_definitions LLAVE_CIERRA
    { $$ = { tipo: 'para', variable: $2, inicio: $4, fin: $6, excluye:
  false, estilos: $8 }; }
  | PARA variable DESDE expr_numerica_for HASTA_EXCLUYE
  expr_numerica_for LLAVE_ABRE style_definitions LLAVE_CIERRA
    { $$ = { tipo: 'para', variable: $2, inicio: $4, fin: $6, excluye:
  true, estilos: $8 }; }
;

style_body
: /* vacio */
  { $$ = []; }
| property_list
  { $$ = $1; }
;

property_list
: property
  { $$ = [$1]; }
| property_list property
  { $1.push($2); $$ = $1; }
;

property
: property_name PUNTO_COMA
  { $$ = $1; }

```

```
| PARA variable DESDE expr_numerica_for HASTA expr_numerica_for
LLAVE_ABRE property_list LLAVE_CIERRA
    { $$ = { tipo: 'para', variable: $2, inicio: $4, fin: $6, excluye:
false, propiedades: $8 }; }
| PARA variable DESDE expr_numerica_for HASTA_EXCLUYE
expr_numerica_for LLAVE_ABRE property_list LLAVE_CIERRA
    { $$ = { tipo: 'para', variable: $2, inicio: $4, fin: $6, excluye:
true, propiedades: $8 }; }
;
```

property_name

```
: ALTO ASIGNAR valor_medida
    { $$ = { nombre: 'height', valor: $3 }; }
| ANCHO ASIGNAR valor_medida
    { $$ = { nombre: 'width', valor: $3 }; }
| MINIMO GUION ANCHO ASIGNAR valor_medida
    { $$ = { nombre: 'min-width', valor: $5 }; }
| MAXIMO GUION ANCHO ASIGNAR valor_medida
    { $$ = { nombre: 'max-width', valor: $5 }; }
| MINIMO GUION ALTO ASIGNAR valor_medida
    { $$ = { nombre: 'min-height', valor: $5 }; }
| MAXIMO GUION ALTO ASIGNAR valor_medida
    { $$ = { nombre: 'max-height', valor: $5 }; }
| FONDO ASIGNAR valor_color
    { $$ = { nombre: 'background-color', valor: $3 }; }
| COLOR ASIGNAR valor_color
    { $$ = { nombre: 'color', valor: $3 }; }
| TEXTO GUION ALINEACION ASIGNAR valor_alineacion
    { $$ = { nombre: 'text-align', valor: $5 }; }
| SIZE ASIGNAR valor_medida
    { $$ = { nombre: 'text-size', valor: $3 }; }
| FUENTE ASIGNAR valor_fuente
    { $$ = { nombre: 'font-family', valor: $3 }; }
| PADDING ASIGNAR valor_medida
    { $$ = { nombre: 'padding', valor: $3 }; }
| PADDING GUION LEFT ASIGNAR valor_medida
    { $$ = { nombre: 'padding-left', valor: $5 }; }
| PADDING GUION RIGHT ASIGNAR valor_medida
    { $$ = { nombre: 'padding-right', valor: $5 }; }
| PADDING GUION TOP ASIGNAR valor_medida
    { $$ = { nombre: 'padding-top', valor: $5 }; }
| PADDING GUION BOTTOM ASIGNAR valor_medida
    { $$ = { nombre: 'padding-bottom', valor: $5 }; }
```

```
| MARGIN ASIGNAR valor_medida
  { $$ = { nombre: 'margin', valor: $3 }; }
| MARGIN GUION LEFT ASIGNAR valor_medida
  { $$ = { nombre: 'margin-left', valor: $5 }; }
| MARGIN GUION RIGHT ASIGNAR valor_medida
  { $$ = { nombre: 'margin-right', valor: $5 }; }
| MARGIN GUION TOP ASIGNAR valor_medida
  { $$ = { nombre: 'margin-top', valor: $5 }; }
| MARGIN GUION BOTTOM ASIGNAR valor_medida
  { $$ = { nombre: 'margin-bottom', valor: $5 }; }
| BORDE ASIGNAR valor_borde
  { $$ = { nombre: 'border', valor: $3 }; }
| BORDE GUION RADIO ASIGNAR valor_medida
  { $$ = { nombre: 'border-radius', valor: $5 }; }
| BORDE RADIO ASIGNAR valor_medida
  { $$ = { nombre: 'border-radius', valor: $4 }; }
| BORDE STYLE ASIGNAR tipo_borde
  { $$ = { nombre: 'border-style', valor: $4 }; }
| BORDE COLOR ASIGNAR valor_color
  { $$ = { nombre: 'border-color', valor: $4 }; }
/* Formas con espacio */
| FONDO COLOR ASIGNAR valor_color
  { $$ = { nombre: 'background-color', valor: $4 }; }
| PADDING TOP ASIGNAR valor_medida
  { $$ = { nombre: 'padding-top', valor: $4 }; }
| PADDING BOTTOM ASIGNAR valor_medida
  { $$ = { nombre: 'padding-bottom', valor: $4 }; }
| PADDING LEFT ASIGNAR valor_medida
  { $$ = { nombre: 'padding-left', valor: $4 }; }
| PADDING RIGHT ASIGNAR valor_medida
  { $$ = { nombre: 'padding-right', valor: $4 }; }
| MARGIN TOP ASIGNAR valor_medida
  { $$ = { nombre: 'margin-top', valor: $4 }; }
| MARGIN BOTTOM ASIGNAR valor_medida
  { $$ = { nombre: 'margin-bottom', valor: $4 }; }
| MARGIN LEFT ASIGNAR valor_medida
  { $$ = { nombre: 'margin-left', valor: $4 }; }
| MARGIN RIGHT ASIGNAR valor_medida
  { $$ = { nombre: 'margin-right', valor: $4 }; }
| TEXTO FUENTE ASIGNAR valor_fuente
  { $$ = { nombre: 'text-font', valor: $4 }; }
| BORDE RIGHT ASIGNAR valor_borde
  { $$ = { nombre: 'border-right', valor: $4 }; }
```

```

| BORDE ANCHO ASIGNAR valor_medida
  { $$ = { nombre: 'border-width', valor: $4 }; }
| BORDE GUION ANCHO ASIGNAR valor_medida
  { $$ = { nombre: 'border-width', valor: $5 }; }
| TEXTO ALINEACION ASIGNAR valor_alineacion
  { $$ = { nombre: 'text-align', valor: $4 }; }
| TEXTO SIZE ASIGNAR valor_medida
  { $$ = { nombre: 'text-size', valor: $4 }; }
;

valor_medida
: expr_numerica_for
  { $$ = $1; }
| PORCENTAJE
  { $$ = { tipo: 'porcentaje', valor: parseFloat($1) }; }
;

valor_color
: COLOR_HEX
  { $$ = { tipo: 'hex', valor: $1 }; }
| RGB PAREN_ABRE expr_numerica_for COMA expr_numerica_for COMA
expr_numerica_for PAREN_CIERRA
  { $$ = { tipo: 'rgb', valor: [$3, $5, $7] }; }
| COLOR_BLUE { $$ = 'blue'; }
| COLOR_WHITE { $$ = 'white'; }
| COLOR_RED { $$ = 'red'; }
| COLOR_GREEN { $$ = 'green'; }
| COLOR_VIOLET { $$ = 'violet'; }
| COLOR_GRAY { $$ = 'gray'; }
| COLOR_BLACK { $$ = 'black'; }
| COLOR_LIGHTGRAY { $$ = 'lightgray'; }
;

valor_alineacion
: CENTER { $$ = 'CENTER'; }
| RIGHT_VALUE { $$ = 'RIGHT'; }
| LEFT_VALUE { $$ = 'LEFT'; }
;

valor_fuente
: HELVETICA { $$ = 'HELVETICA'; }
| SANS { $$ = 'SANS'; }
| SANS SERIF { $$ = 'SANS SERIF'; }

```

```

| SERIF      { $$ = 'SERIF'; }
| MONO       { $$ = 'MONO'; }
| CURSIVE    { $$ = 'CURSIVE'; }
;

valor_borde
: valor_medida tipo_borde valor_color
  { $$ = { ancho: $1, estilo: $2, color: $3 }; }
;

tipo_borde
: DOTTED { $$ = 'DOTTED'; }
| LINE   { $$ = 'LINE'; }
| DOUBLE { $$ = 'DOUBLE'; }
| SOLID  { $$ = 'SOLID'; }
;

numero
: ENTERO
  { $$ = parseInt($1); }
| DECIMAL
  { $$ = parseFloat($1); }
;

variable
: VARIABLE
  { $$ = $1; }
;

expr_numerica_for
: termino_numerico_for
  { $$ = $1; }
| expr_numerica_for SUMA termino_numerico_for
  { $$ = { op: '+', left: $1, right: $3 }; }
| expr_numerica_for GUION termino_numerico_for
  { $$ = { op: '-', left: $1, right: $3 }; }
| expr_numerica_for MODULO termino_numerico_for
  { $$ = { op: '%', left: $1, right: $3 }; }
| expr_numerica_for IGUALDAD termino_numerico_for
  { $$ = { op: '==', left: $1, right: $3 }; }
;

termino_numerico_for

```

```

: factor_numerico_for
  { $$ = $1; }
| termino_numerico_for MULTIPLICADOR factor_numerico_for
  { $$ = { op: '*', left: $1, right: $3 }; }
| termino_numerico_for SLASH factor_numerico_for
  { $$ = { op: '/', left: $1, right: $3 }; }
;

factor_numerico_for
: numero
  { $$ = $1; }
| variable
  { $$ = $1; }
| PAREN_ABRE expr_numerica_for PAREN_CIERRA
  { $$ = $2; }
;

```

Análisis de Gramática Principal para el Analizador Léxico

Para el analizador Léxico utilizamos las bondades que la librería Jison nos ofrece y por tanto a continuación se mostraran las expresiones regulares que se utilizaron para el lexer implementado.

```

/*
 * Analizador Lexico
 */

%{
  var erroresLexicos = [];
  var erroresSintacticos = [];
  parser.erroresLexicos = erroresLexicos;
  parser.erroresSintacticos = erroresSintacticos;

  function registrarErrorLexico(lexema, linea, columna){

```

```

erroresLexicos.push({
    tipo: 'lexico',
    lexema: lexema,
    linea: linea,
    columna: columna,
    mensaje: 'Token no reconocido: ' + lexema
});
}

function registrarErrorSintactico(mensaje, lexema, linea, columna){
    erroresSintacticos.push({
        tipo: 'sintactico',
        lexema: lexema,
        linea: linea,
        columna: columna,
        mensaje: mensaje
    });
}

function registrarErrorSintacticoActual(mensaje) {
    var linea = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
    var columna = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_column || 0) : 0;
    var lexema = (typeof yytext !== 'undefined') ? yytext : '';
    registrarErrorSintactico(mensaje, lexema, linea, columna);
}

parser.parseError = function(str, hash) {
    registrarErrorSintactico(str, hash.text || hash.token, hash.line +
1, (hash.loc ? hash.loc.first_column : 0));
    if (!hash.recoverable) {
        throw new Error(str);
    }
};
%}

%lex

%options ranges yylineno

%%

\s+          /* ignorar espacios y saltos */

```



```

[\u200B\u200C\u200D\uFEFF\u00A0]+ /* ignorar invisibles y nbsp */
\/\*[\s\S]*?\*\/ /* ignorar comentarios de bloque */
"#".* /* ignorar comentarios de linea */

"import"          return 'IMPORTAR';
"execute"         return 'EJECUTAR';
"load"           return 'CARGAR';
"render"         return 'RENDER';
"main"           return 'MAIN';
"function"       return 'FUNCION';
"while"          return 'MIENTRAS';
"do"             return 'DO';
"switch"         return 'SWITCH';
"case"           return 'CASO';
"default"        return 'DEFECTO';
"break"          return 'BREAK';
"for"            return 'PARA';
"if"             return 'SI';
"else"           return 'SINO';

"int"            return 'TIPO_ENTERO';
"float"          return 'TIPO_FLOTANTE';
"string"         return 'TIPO_CADENA';
"boolean"        return 'TIPO_BOOLEANO';
"char"           return 'TIPO_CARACTER';

"True"           return 'VERDADERO';
"False"          return 'FALSO';
"true"           return 'VERDADERO';
"false"          return 'FALSO';

"=="            return 'IGUALDAD';
"!="            return 'NO_IGUAL';
"<="           return 'MENOR_IGUAL';
">="           return 'MAYOR_IGUAL';
"<"           return 'MENOR';
">"           return 'MAYOR';
"&&"          return 'AND';
"||"            return 'OR';
"++"            return 'INCREMENTO';

";"             return 'PUNTO_COMA';
","            return 'COMA';

```

```

"="                return 'ASIGNACION';
"("                return 'PARENTESIS_ABIERTO';
")"                return 'PARENTESIS_CERRADO';
"{"                return 'LLAVE_ABIERTA';
"}"                return 'LLAVE_CERRADA';
"["                return 'CORCHETE_ABIERTO';
"]"                return 'CORCHETE_CERRADO';
"@"                return 'DECORADOR';
"+"                return 'SUMA';
"_"                return 'RESTA';
"*"                return 'MULTIPLICACION';
"/"                return 'DIVISION';
":"                return 'DOS_PUNTOS';
"."                return 'PUNTO';

"$"[a-zA-Z_][a-zA-Z0-9_]*    return 'VARIABLE';

"^"^[^`]*`"            return 'CONSULTA_DB';
\"^[^\"]*\"            return 'CADENA';
\'^[^\']*\'            return 'CARACTER';
[0-9]+\".\"[0-9]+        return 'DECIMAL';
[0-9]+                return 'ENTERO';
[a-zA-Z\u00C0-\u017F_][a-zA-Z0-9_\u00C0-\u017F]*
return 'IDENTIFICADOR';

<<EOF>>                return 'EOF';

. {
    var linea = (typeof yyloc !== 'undefined' && yyloc) ?
(yyloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
    var columna = (typeof yyloc !== 'undefined' && yyloc) ?
(yyloc.first_column || 0) : 0;
    registrarErrorLexico(yytext, linea, columna);
}

/lex

```

Análisis de Gramática Principal para el Analizador Sintáctico

Nuevamente Jison nos permite utilizarlo para generar nuestro analizador sintáctico permitiendo la declaración de terminales y no terminales, asimismo permite la estructuración de reglas de producción que nuestro lenguaje requiere.

```
/*
 * Analizador Sintactico
 */

%start programa

%%

programa
: lista_imports lista_declaraciones lista_funciones bloque_main EOF
  { $$ = { imports: $1, declaraciones: $2, funciones: $3, main: $4 };
return $$; }
;

lista_imports
: /* vacio */
  { $$ = []; }
| lista_imports import_stmt
  { $$ = $2 ? $1.concat([$2]) : $1; }
| lista_imports error PUNTO_COMA {
  registrarErrorSintacticoActual('Import invalido');
  yyerrok;
  $$ = $1;
}
;

import_stmt
: IMPORTAR CADENA PUNTO_COMA
  { $$ = { tipo: 'import', ruta: $2 }; }
| IMPORTAR error PUNTO_COMA {
  registrarErrorSintacticoActual('Import incompleto');
  yyerrok;
  $$ = null;
}
```

```

    }
;

lista_declaraciones
: /* vacio */
  { $$ = []; }
| lista_declaraciones declaracion
  { $$ = $1.concat([$2]); }
| lista_declaraciones error PUNTO_COMA {
    registrarErrorSintacticoActual('Declaracion invalida');
    yyerrok;
    $$ = $1;
  }
;

lista_funciones
: /* vacio */
  { $$ = []; }
| lista_funciones funcion
  { $$ = $2 ? $1.concat([$2]) : $1; }
| lista_funciones error LLAVE_CERRADA {
    registrarErrorSintacticoActual('Funcion invalida');
    yyerrok;
    $$ = $1;
  }
;

bloque_main
: MAIN LLAVE_ABIERTA lista_sentencias_main LLAVE_CERRADA
  { $$ = { tipo: 'main', sentencias: $3 }; }
| MAIN error LLAVE_CERRADA {
    registrarErrorSintacticoActual('Bloque main invalido');
    yyerrok;
    $$ = { tipo: 'main', sentencias: [] };
  }
;

function
: FUNCION IDENTIFICADOR PARENTESIS_ABIERTO PARENTESIS_CERRADO
LLAVE_ABIERTA lista_sentencias_funcion LLAVE_CERRADA
  { $$ = { tipo: 'funcion', nombre: $2, parametros: [], sentencias: $6
}; }

```

```

| FUNCION IDENTIFICADOR PARENTESIS_ABIERTO lista_parametros
PARENTESIS_CERRADO LLAVE_ABIERTA lista_sentencias_funcion LLAVE_CERRADA
  { $$ = { tipo: 'funcion', nombre: $2, parametros: $4, sentencias: $7
}; }

| FUNCION error LLAVE_CERRADA {
  registrarErrorSintacticoActual('Funcion invalida');
yyerrok;
  $$ = null;
}

;

lista_parametros
: parametro
  { $$ = [$1]; }
| lista_parametros COMA parametro
  { $$ = $1.concat([$3]); }
;

parametro
: tipo IDENTIFICADOR
  { $$ = { tipo: $1, id: $2 }; }
;

tipo
: TIPO_ENTERO
  { $$ = 'int'; }
| TIPO_FLOTANTE
  { $$ = 'float'; }
| TIPO_CADENA
  { $$ = 'string'; }
| TIPO_BOOLEANO
  { $$ = 'boolean'; }
| TIPO_CARACTER
  { $$ = 'char'; }
;

declaracion
: tipo IDENTIFICADOR ASIGNACION expresion PUNTO_COMA
  { $$ = { tipo: 'declaracion', dato: $1, id: $2, valor: $4 }; }
| tipo CORCHETE_ABIERTO CORCHETE_CERRADO IDENTIFICADOR ASIGNACION
arreglo_tamano PUNTO_COMA
  { $$ = { tipo: 'arreglo_tamano', dato: $1, id: $4, valor: $6 }; }

```

```

    | tipo CORCHETE_ABIERTO CORCHETE_CERRADO IDENTIFICADOR ASIGNACION
arreglo_literal PUNTO_COMA
    { $$ = { tipo: 'arreglo_literal', dato: $1, id: $4, valor: $6 }; }
    | tipo CORCHETE_ABIERTO CORCHETE_CERRADO IDENTIFICADOR ASIGNACION
EJECUTAR CONSULTA_DB PUNTO_COMA
    { $$ = { tipo: 'arreglo_execute', dato: $1, id: $4, consulta: $7 }; }
}

| RENDER IDENTIFICADOR PARENTESIS_ABIERTO PARENTESIS_CERRADO
PUNTO_COMA
    { $$ = { tipo: 'render', invocacion: { componente: $2, args: [] } }; }
}

| RENDER IDENTIFICADOR PARENTESIS_ABIERTO lista_expresiones
PARENTESIS_CERRADO PUNTO_COMA
    { $$ = { tipo: 'render', invocacion: { componente: $2, args: $4 } }; }
}

;

arreglo_tamano
: CORCHETE_ABIERTO ENTERO CORCHETE_CERRADO
    { $$ = { size: Number($2) }; }
;

arreglo_literal
: LLAVE_ABIERTA lista_expresiones LLAVE_CERRADA
    { $$ = $2; }
;

lista_expresiones
: expresion
    { $$ = [$1]; }
| lista_expresiones COMA expresion
    { $$ = $1.concat([$3]); }
;

lista_sentencias_funcion
: /* vacio */
    { $$ = []; }
| lista_sentencias_funcion sentencia_funcion
    { $$ = $1.concat([$2]); }
| lista_sentencias_funcion error PUNTO_COMA {
    registrarErrorSintacticoActual('Sentencia invalida dentro de
funcion');
    yyerrok;
}

```

```

    $$ = $1;
}
;

sentencia_funcion
: EJECUTAR CONSULTA_DB PUNTO_COMA
  { $$ = { tipo: 'execute', consulta: $2 }; }
| CARGAR expresion PUNTO_COMA
  { $$ = { tipo: 'load', valor: $2 }; }
;

lista_sentencias_main
: /* vacio */
  { $$ = []; }
| lista_sentencias_main sentencia_main
  { $$ = $1.concat($2 ? [$2] : []); }
| lista_sentencias_main error PUNTO_COMA {
  registrarErrorSintacticoActual('Sentencia invalida en main');
  yyerrok;
  $$ = $1;
}
;

sentencia_main
: invocacion_componente PUNTO_COMA
  { $$ = $1; }
| asignacion PUNTO_COMA
  { $$ = $1; }
| ciclo_while
  { $$ = $1; }
| ciclo_for
  { $$ = $1; }
| ciclo_do
  { $$ = $1; }
| sentencia_switch
  { $$ = $1; }
| condicional
  { $$ = $1; }
| BREAK PUNTO_COMA
  { $$ = { tipo: 'break' }; }
;

ciclo_do

```

```

: DO LLAVE_ABIERTA lista_sentencias_main LLAVE_CERRADA MIENTRAS
PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO
  { $$ = { tipo: 'do_while', cuerpo: $3, condicion: $7 }; }
;

sentencia_switch
: SWITCH PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO LLAVE_ABIERTA
lista_casos LLAVE_CERRADA
  { $$ = { tipo: 'switch', expresion: $3, casos: $6 }; }
;

lista_casos
: /* vacio */
  { $$ = []; }
| lista_casos caso
  { $$ = $1.concat([$2]); }
;

caso
: CASO expresion DOS_PUNTOS lista_sentencias_main
  { $$ = { tipo: 'case', valor: $2, cuerpo: $4 }; }
| DEFECTO DOS_PUNTOS lista_sentencias_main
  { $$ = { tipo: 'default', cuerpo: $3 }; }
;

invocacion_componente
: DECORADOR IDENTIFICADOR PARENTESIS_ABIERTO PARENTESIS_CERRADO
  { $$ = { tipo: 'invocacion', componente: $2, args: [] }; }
| DECORADOR IDENTIFICADOR PARENTESIS_ABIERTO lista_expresiones
PARENTESIS_CERRADO
  { $$ = { tipo: 'invocacion', componente: $2, args: $4 }; }
;

asignacion
: IDENTIFICADOR ASIGNACION expresion
  { $$ = { tipo: 'asignacion', target: { tipo: 'id', valor: $1 },
valor: $3 }; }
| acceso_arreglo ASIGNACION expresion
  { $$ = { tipo: 'asignacion', target: $1, valor: $3 }; }
;

ciclo_while

```



```

: MIENTRAS PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO
LLAVE_ABIERTA lista_sentencias_main LLAVE_CERRADA
    { $$ = { tipo: 'while', condicion: $3, cuerpo: $6 }; }
;

ciclo_for
: PARA PARENTESIS_ABIERTO asignacion PUNTO_COMA expresion PUNTO_COMA
paso_for PARENTESIS_CERRADO LLAVE_ABIERTA lista_sentencias_main
LLAVE_CERRADA
    { $$ = { tipo: 'for', init: $3, condicion: $5, paso: $7, cuerpo: $10
}; }
;

paso_for
: asignacion
    { $$ = $1; }
| IDENTIFICADOR ASIGNACION IDENTIFICADOR INCREMENTO
    { $$ = { tipo: 'post_increment_assign', id: $1, valor: $3 }; }
| IDENTIFICADOR INCREMENTO
    { $$ = { tipo: 'post_increment', id: $1 }; }
;

condicional
: SI PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO LLAVE_ABIERTA
lista_sentencias_main LLAVE_CERRADA
    { $$ = { tipo: 'if', condicion: $3, then: $6, else: null }; }
| SI PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO LLAVE_ABIERTA
lista_sentencias_main LLAVE_CERRADA SINO LLAVE_ABIERTA
lista_sentencias_main LLAVE_CERRADA
    { $$ = { tipo: 'if', condicion: $3, then: $6, else: $10 }; }
;

expresion
: expr_logica
    { $$ = $1; }
;

expr_logica
: expr_relacional
    { $$ = $1; }
| expr_logica AND expr_relacional
    { $$ = { op: '&&', left: $1, right: $3 }; }
| expr_logica OR expr_relacional

```

```
    { $$ = { op: '||', left: $1, right: $3 }; }  
;  

```

expr_relacional

```
: expr_aritmetica  
  { $$ = $1; }  
| expr_aritmetica IGUALDAD expr_aritmetica  
  { $$ = { op: '==', left: $1, right: $3 }; }  
| expr_aritmetica NO_IGUAL expr_aritmetica  
  { $$ = { op: '!=', left: $1, right: $3 }; }  
| expr_aritmetica MENOR expr_aritmetica  
  { $$ = { op: '<', left: $1, right: $3 }; }  
| expr_aritmetica MAYOR expr_aritmetica  
  { $$ = { op: '>', left: $1, right: $3 }; }  
| expr_aritmetica MENOR_IGUAL expr_aritmetica  
  { $$ = { op: '<=', left: $1, right: $3 }; }  
| expr_aritmetica MAYOR_IGUAL expr_aritmetica  
  { $$ = { op: '>=', left: $1, right: $3 }; }  
;  

```

expr_aritmetica

```
: termino  
  { $$ = $1; }  
| expr_aritmetica SUMA termino  
  { $$ = { op: '+', left: $1, right: $3 }; }  
| expr_aritmetica RESTA termino  
  { $$ = { op: '-', left: $1, right: $3 }; }  
;  

```

termino

```
: factor  
  { $$ = $1; }  
| termino MULTIPLICACION factor  
  { $$ = { op: '*', left: $1, right: $3 }; }  
| termino DIVISION factor  
  { $$ = { op: '/', left: $1, right: $3 }; }  
;  

```

factor

```
: ENTERO  
  { $$ = Number($1); }  
| DECIMAL  
  { $$ = Number($1); }
```

```

| CADENA
  { $$ = $1; }
| CARACTER
  { $$ = $1; }
| VERDADERO
  { $$ = true; }
| FALSO
  { $$ = false; }
| VARIABLE
  { $$ = { tipo: 'id', valor: $1.substring(1) }; }
| IDENTIFICADOR
  { $$ = { tipo: 'id', valor: $1 }; }
| acceso_arreglo
  { $$ = $1; }
| PARENTESIS_ABIERTO expresion PARENTESIS_CERRADO
  { $$ = $2; }
;

acceso_arreglo
: IDENTIFICADOR CORCHETE_ABIERTO expresion CORCHETE_CERRADO
  { $$ = { tipo: 'acceso_arreglo', id: $1, indice: $3 }; }
;

```

Análisis de Gramática Base Datos para el Analizador Léxico

```

/*
 * Analizador Lexico
 */

%{
  var erroresLexicos = [];
  var erroresSintacticos = [];
  parser.erroresLexicos = erroresLexicos;
  parser.erroresSintacticos = erroresSintacticos;

```

```

function registrarErrorLexico(lexema, linea, columna){
  erroresLexicos.push({
    tipo: 'lexico',
    lexema: lexema,
    linea: linea,
    columna: columna,
    mensaje: 'Token no reconocido: ' + lexema
  });
}

function registrarErrorSintactico(mensaje, lexema, linea, columna){
  erroresSintacticos.push({
    tipo: 'sintactico',
    lexema: lexema,
    linea: linea,
    columna: columna,
    mensaje: mensaje
  });
}

function registrarErrorSintacticoActual(mensaje){
  var linea = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
  var columna = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_column || 0) : 0;
  var lexema = (typeof yytext !== 'undefined') ? yytext : '';
  registrarErrorSintactico(mensaje, lexema, linea, columna);
}

parser.parseError = function(str, hash) {
  registrarErrorSintactico(str, hash.text || hash.token, hash.line +
1, hash.loc ? hash.loc.first_column : 0);
  if (!hash.recoverable) {
    throw new Error(str);
  }
};
%}

%lex
%options ranges yylineno
%%

```

```

\s+                                /* ignorar espacios y saltos */
[\u200B\u200C\u200D\uFEFF\u00A0]+ /* ignorar invisibles y nbsp */
\/\*[\s\S]*?\*\/                 /* ignorar comentarios de bloque */
"#".*                             /* ignorar comentarios de línea */
"--".*                            /* ignorar comentarios SQL estilo -- */

"TABLE"                           return 'TABLA';
"COLUMNS"                        return 'COLUMNAS';
"IN"                             return 'EN';
"DELETE"                         return 'BORRAR';

"int"                             return 'TIPO_ENTERO';
"float"                          return 'TIPO_FLOTANTE';
"string"                         return 'TIPO_CADENA';
"boolean"                        return 'TIPO_BOOLEANO';
"char"                           return 'TIPO_CARACTER';

"true"                           return 'VERDADERO';
"false"                          return 'FALSO';

","                             return 'COMA';
"="                             return 'ASIGNACION';
"."                             return 'PUNTO';
";"                             return 'PUNTO_COMA';
"["                             return 'CORCHETE_ABRE';
"]"                             return 'CORCHETE_CIERRA';

\"[^\"]*"                        return 'CADENA';
\'[^\']*\'                       return 'CARACTER';
[0-9]+\."[0-9]+                 return 'DECIMAL';
[0-9]+                           return 'ENTERO';
[a-zA-Z\u00C0-\u017F_][a-zA-Z0-9_\u00C0-\u017F]*
return 'IDENTIFICADOR';

<<EOF>>                         return 'EOF';

. {
    var linea = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
    var columna = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_column || 0) : 0;

```

```
    registrarErrorLexico(yytext, linea, columna);
}

/lex
```

Análisis de Gramática Base Datos para el Analizador Sintáctico

```
/*
 * Analizador Sintactico
 */

%start programa

%%

programa
: lista_sentencias EOF
  { return $1; }
;

lista_sentencias
: /* vacio */
  { $$ = []; }
| lista_sentencias sentencia
  { $$ = $1.concat($2 ? [$2] : []); return $$; }
;

sentencia
: crear_tabla PUNTO_COMA
  { $$ = $1; }
| seleccionar_columna PUNTO_COMA
  { $$ = $1; }
| insertar_registro
  { $$ = $1; }
| actualizar_registro PUNTO_COMA
```

```

    { $$ = $1; }
| eliminar_registro PUNTO_COMA
    { $$ = $1; }
| error PUNTO_COMA {
    registrarErrorSintacticoActual('Sentencia DB invalida');
    yyerrok;
    $$ = null;
}
;

```

crear_tabla

```

: TABLA IDENTIFICADOR COLUMNAS lista_columnas
    { $$ = { tipo: 'create_table', tabla: $2, columnas: $4 }; }
| TABLA error COLUMNAS lista_columnas {
    registrarErrorSintacticoActual('Creacion de tabla invalida');
    yyerrok;
    $$ = { tipo: 'create_table', tabla: null, columnas: $4 };
}
;

```

lista_columnas

```

: columna_def
    { $$ = [$1]; }
| lista_columnas COMA columna_def
    { $$ = $1.concat([$3]); }
;

```

columna_def

```

: IDENTIFICADOR ASIGNACION tipo_dato
    { $$ = { nombre: $1, tipo: $3 }; }
;

```

tipo_dato

```

: TIPO_ENTERO
    { $$ = 'int'; }
| TIPO_FLOTANTE
    { $$ = 'float'; }
| TIPO_CADENA
    { $$ = 'string'; }
| TIPO_BOOLEANO
    { $$ = 'boolean'; }
| TIPO_CARACTER
    { $$ = 'char'; }

```

```

;

seleccionar_columna
: IDENTIFICADOR PUNTO lista_campos_seleccion
{
  $$ = {
    tipo: 'select_columns',
    tabla: $1,
    columnas: $3,
  };
}
| IDENTIFICADOR error lista_campos_seleccion {
  registrarErrorSintacticoActual('Consulta de columna invalida');
yyerrok;
  $$ = {
    tipo: 'select_columns',
    tabla: $1,
    columnas: $3,
  };
}
;

lista_campos_seleccion
: IDENTIFICADOR
{ $$ = [$1]; }
| lista_campos_seleccion COMA IDENTIFICADOR
{ $$ = $1.concat([$3]); }
;

insertar_registro
: IDENTIFICADOR CORCHETE_ABRE lista_asignaciones CORCHETE_CIERRA
{ $$ = { tipo: 'insert', tabla: $1, valores: $3 }; }
| IDENTIFICADOR CORCHETE_ABRE lista_asignaciones CORCHETE_CIERRA
PUNTO_COMA
{ $$ = { tipo: 'insert', tabla: $1, valores: $3 }; }
| IDENTIFICADOR CORCHETE_ABRE error CORCHETE_CIERRA {
  registrarErrorSintacticoActual('Insercion de registro invalida');
yyerrok;
  $$ = { tipo: 'insert', tabla: $1, valores: [] };
}
;

actualizar_registro

```



```

: IDENTIFICADOR CORCHETE_ABRE lista_asignaciones CORCHETE_CIERRA EN
ENTERO
  { $$ = { tipo: 'update', tabla: $1, valores: $3, id: Number($6) }; }
| IDENTIFICADOR CORCHETE_ABRE error CORCHETE_CIERRA EN ENTERO {
  registrarErrorSintacticoActual('Actualizacion de registro
invalida');
  yyerrok;
  $$ = { tipo: 'update', tabla: $1, valores: [], id: Number($6) };
}
;

eliminar_registro
: IDENTIFICADOR BORRAR ENTERO
  { $$ = { tipo: 'delete', tabla: $1, id: Number($3) }; }
| IDENTIFICADOR BORRAR error {
  registrarErrorSintacticoActual('Eliminacion de registro
invalida');
  yyerrok;
  $$ = { tipo: 'delete', tabla: $1, id: null };
}
;

lista_asignaciones
: asignacion
  { $$ = [$1]; }
| lista_asignaciones COMA asignacion
  { $$ = $1.concat([$3]); }
;

asignacion
: IDENTIFICADOR ASIGNACION valor
  { $$ = { columna: $1, valor: $3 }; }
;

valor
: CADENA
  { $$ = $1.slice(1, -1); }
| CARACTER
  { $$ = $1.slice(1, -1); }
| DECIMAL
  { $$ = Number($1); }
| ENTERO
  { $$ = Number($1); }

```

```
| VERDADERO
  { $$ = true; }
| FALSO
  { $$ = false; }
;
```

Análisis de Gramática Componentes para el Analizador Léxico

```
/*
 * Analizador Lexico
 */

%{
  var yyerrok = 0, yyclearin = 0;
  var erroresLexicos = [];
  var erroresSintacticos = [];
  parser.erroresLexicos = erroresLexicos;
  parser.erroresSintacticos = erroresSintacticos;

  function registrarErrorLexico(lexema, linea, columna){
    erroresLexicos.push({
      tipo: 'lexico',
      lexema: lexema,
      linea: linea,
      columna: columna,
      mensaje: 'Token no reconocido: ' + lexema
    });
  }

  function registrarErrorSintactico(mensaje, lexema, linea, columna){
    erroresSintacticos.push({
      tipo: 'sintactico',
      lexema: lexema,
      linea: linea,
      columna: columna,
      mensaje: mensaje
    });
  }
}
```

```

    });
}

function registrarErrorSintacticoActual(mensaje) {
    var linea = (typeof yyloc !== 'undefined' && yyloc) ?
(yyloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
    var columna = (typeof yyloc !== 'undefined' && yyloc) ?
(yyloc.first_column || 0) : 0;
    var lexema = (typeof yytext !== 'undefined') ? yytext : '';
    registrarErrorSintactico(mensaje, lexema, linea, columna);
}

parser.parseError = function(str, hash) {
    registrarErrorSintactico(str, hash.text || hash.token, hash.line +
1, (hash.loc ? hash.loc.first_column : 0));
    if (!hash.recoverable) {
        throw new Error(str);
    }
};
%}

%lex
%options ranges yylineno
%%

\s+                                /* ignorar espacios y saltos */
[\u200B\u200C\u200D\uFEFF\u00A0]+ /* ignorar invisibles y nbsp */
\/\*[\s\S]*?\*\/                  /* ignorar comentarios de bloque */
"#".*                             /* ignorar comentarios de línea */

\[\'[^\']*\'                      return 'CARACTER';
/* Palabras reservadas */

"component"      return 'COMPONENTE';
"int"            return 'TIPO_ENTERO';
"float"          return 'TIPO_FLOTANTE';
"string"         return 'TIPO_CADENA';
"boolean"        return 'TIPO_BOOLEANO';
"char"           return 'TIPO_CARACTER';
"function"       return 'FUNCION';
"T"              return 'TEXTO';
"IMG"            return 'IMAGEN';

```

```
"FORM"           return 'FORMULARIO';
"INPUT_TEXT"     return 'INPUT_TEXTO';
"INPUT_NUMBER"   return 'INPUT_NUMERO';
"INPUT_BOOL"     return 'INPUT_BOOLEANO';
"SUBMIT"         return 'ENVIAR';
"id"             return 'ID_FORMULARIO';
"label"          return 'ETIQUETA_FORMULARIO';
"value"          return 'VALOR_FORMULARIO';
"for"            return 'PARA';
"each"           return 'CADA';
"track"          return 'SEGUIMIENTO';
"empty"          return 'VACIO';
"if"             return 'SI';
"else"           return 'SINO';
"while"          return 'MIENTRAS';
"switch"         return 'SWITCH';
"Switch"         return 'SWITCH';
"case"           return 'CASO';
"default"        return 'DEFECTO';
"true"           return 'VERDADERO';
"false"          return 'FALSO';

"{"             return 'LLAVE_ABRE';
"}"             return 'LLAVE_CIERRA';
"("             return 'PARENTESIS_ABRE';
")"             return 'PARENTESIS_CIERRA';
"["             return 'TABLA_ABRE';
"]"             return 'TABLA_CIERRA';
"["             return 'CORCHETE_ABRE';
"]"             return 'CORCHETE_CIERRA';
"<"            return 'MENOR';
">"            return 'MAYOR';
","            return 'COMA';
"@"[a-zA-Z_][a-zA-Z0-9_]* return 'REFERENCIA_CAMPO';
"@"            return 'REFERENCIA_ID_FORMULARIO';
":"           return 'DOS_PUNTOS';
"="           return 'ASIGNAR';
"=="          return 'IGUALDAD';
"!="          return 'NO_IGUAL';
"_"           return 'GUION';
"+"           return 'SUMA';
```

```

"*"          return 'MULTIPLICADOR';
"/"          return 'DIVISION';

"$"[a-zA-Z_][a-zA-Z0-9_]*      return 'VARIABLE';
\"[^\"]*\"      return 'CADENA';
\'[^\']*\'      return 'CARACTER';
"'"          return 'COMILLA_SIMPLE';
"\""          return 'COMILLA_DOBLE';
[0-9]+("."[0-9]+)?      return 'NUMERO';
[a-zA-Z_][a-zA-Z0-9_-]*      return 'IDENTIFICADOR';

"&&"          return 'AND';
"||"          return 'OR';

<<EOF>>      return 'EOF';

. {
  var linea = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_line || yylineno || 0) : ((typeof yylineno !==
'undefined') ? yylineno : 0);
  var columna = (typeof yylloc !== 'undefined' && yylloc) ?
(yylloc.first_column || 0) : 0;
  registrarErrorLexico(yytext, linea, columna);
}

/lex

```

Análisis de Gramática Componentes para el Analizador Sintáctico

```

/*
 * Analizador Sintactico
 */

%start programa
%%

programa
: lista_componentes EOF
  { return $1; }
;

lista_componentes
: /* vacío */
  { $$ = []; }
| lista_componentes componente
  { $$ = $1.concat([$2]); return $$; }
| lista_componentes error LLAVE_CIERRA {
  registrarErrorSintacticoActual('Componente invalido');
  yyerrok;
  $$ = $1; return $$;
}
;

componente
: COMPONENTE PARENTESIS_ABRE PARENTESIS_CIERRA LLAVE_ABRE
lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'componente', nombre: 'ComponenteAnonimo',
parametros: [], elementos: $5 }; }
| COMPONENTE PARENTESIS_ABRE lista_parametros PARENTESIS_CIERRA
LLAVE_ABRE lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'componente', nombre: 'ComponenteAnonimo',
parametros: $3, elementos: $6 }; }
| IDENTIFICADOR PARENTESIS_ABRE PARENTESIS_CIERRA LLAVE_ABRE
lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'componente', nombre: $1, parametros: [], elementos:
$5 }; }
| IDENTIFICADOR PARENTESIS_ABRE lista_parametros PARENTESIS_CIERRA
LLAVE_ABRE lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'componente', nombre: $1, parametros: $3, elementos:
$6 }; }
| COMPONENTE error LLAVE_CIERRA {

```

```

        registrarErrorSintacticoActual('Declaracion de componente
invalida');
        yyerrok;
        $$ = { tipo: 'componente', parametros: [], elementos: [] };
    }
;

lista_parametros
: parametro
  { $$ = [$1]; }
| lista_parametros COMA parametro
  { $$ = $1.concat([$3]); }
;

parametro
: tipo IDENTIFICADOR
  { $$ = { tipo: $1, id: $2 }; }
;

tipo
: TIPO_ENTERO
  { $$ = 'int'; }
| TIPO_FLOTANTE
  { $$ = 'float'; }
| TIPO_CADENA
  { $$ = 'string'; }
| TIPO_BOOLEANO
  { $$ = 'boolean'; }
| TIPO_CARACTER
  { $$ = 'char'; }
| FUNCION
  { $$ = 'function'; }
;

lista_elementos
: /* vacío */
  { $$ = []; }
| lista_elementos elemento
  { $$ = $1.concat([$2]); }
| lista_elementos error {
    registrarErrorSintacticoActual('Elemento de componente invalido');
    yyerrok;
    $$ = $1;
}

```

```

    }
;

elemento
: seccion
  { $$ = $1; }
| tabla
  { $$ = $1; }
| texto
  { $$ = $1; }
| imagen
  { $$ = $1; }
| formulario
  { $$ = $1; }
| logica
  { $$ = $1; }
;

seccion
: CORCHETE_ABRE lista_elementos CORCHETE_CIERRA
  { $$ = { tipo: 'seccion', estilos: [], elementos: $2 }; }
| MENOR lista_estilos MAYOR CORCHETE_ABRE lista_elementos
CORCHETE_CIERRA
  { $$ = { tipo: 'seccion', estilos: $2, elementos: $5 }; }
| MENOR error CORCHETE_CIERRA {
  registrarErrorSintacticoActual('Seccion invalida');
  yyerrok;
  $$ = { tipo: 'seccion', estilos: [], elementos: [] };
}
;

lista_estilos
: IDENTIFICADOR
  { $$ = [$1]; }
| lista_estilos COMA IDENTIFICADOR
  { $$ = $1.concat([$3]); }
;

tabla
: TABLA_ABRE tabla_contenido TABLA_CIERRA
  { $$ = { tipo: 'tabla', estilos: [], filas: $2 }; }
| MENOR lista_estilos MAYOR TABLA_ABRE tabla_contenido TABLA_CIERRA
  { $$ = { tipo: 'tabla', estilos: $2, filas: $5 }; }

```



```

| TABLA_ABRE error TABLA_CIERRA {
    registrarErrorSintacticoActual('Tabla invalida');
    yyerrok;
    $$ = { tipo: 'tabla', estilos: [], filas: [] };
}
;

tabla_contenido
: /* vacío */
{ $$ = []; }
| tabla_contenido fila_tabla
{ $$ = $1.concat([$2]); }
;

fila_tabla
: TABLA_ABRE celda_tabla TABLA_CIERRA
{ $$ = { tipo: 'fila', celdas: $2 }; }
;

celda_tabla
: /* vacío */
{ $$ = []; }
| celda_tabla TABLA_ABRE lista_elementos TABLA_CIERRA
{ $$ = $1.concat([{ tipo: 'celda', elementos: $3 }]); }
;

texto
: TEXTO PARENTESIS_ABRE expresion PARENTESIS_CIERRA
{ $$ = { tipo: 'texto', estilos: [], contenido: $3 }; }
| TEXTO MENOR lista_estilos MAYOR PARENTESIS_ABRE expresion
PARENTESIS_CIERRA
{ $$ = { tipo: 'texto', estilos: $3, contenido: $6 }; }
| TEXTO error PARENTESIS_CIERRA {
    registrarErrorSintacticoActual('Texto invalido');
    yyerrok;
    $$ = { tipo: 'texto', estilos: [], contenido: '' };
}
;

imagen
: IMAGEN PARENTESIS_ABRE lista_fuentes_imagen PARENTESIS_CIERRA
{ $$ = { tipo: 'imagen', estilos: [], fuentes: $3 }; }

```

```

| IMAGEN MENOR lista_estilos MAYOR PARENTESIS_ABRE
lista_fuentes_imagen PARENTESIS_CIERRA
{ $$ = { tipo: 'imagen', estilos: $3, fuentes: $6 }; }
| IMAGEN error PARENTESIS_CIERRA {
    registrarErrorSintacticoActual('Imagen invalida');
    yyerrok;
    $$ = { tipo: 'imagen', estilos: [], fuentes: [] };
}
;

lista_fuentes_imagen
: fuente_imagen
{ $$ = [$1]; }
| lista_fuentes_imagen COMA fuente_imagen
{ $$ = $1.concat([$3]); }
;

fuente_imagen
: CADENA
{ $$ = { tipo: 'cadena', valor: $1 }; }
| VARIABLE
{ $$ = { tipo: 'variable', valor: $1 }; }
| VARIABLE CORCHETE_ABRE NUMERO CORCHETE_CIERRA
{ $$ = { tipo: 'acceso_arreglo', variable: $1, indice: Number($3) }; }
}
;

formulario
: FORMULARIO LLAVE_ABRE lista_inputs LLAVE_CIERRA submit_opcional
{ $$ = { tipo: 'formulario', estilos: [], elementos: $3, submit: $5
}; }
| FORMULARIO MENOR lista_estilos MAYOR LLAVE_ABRE lista_inputs
LLAVE_CIERRA submit_opcional
{ $$ = { tipo: 'formulario', estilos: $3, elementos: $6, submit: $8
}; }
| FORMULARIO error LLAVE_CIERRA {
    registrarErrorSintacticoActual('Formulario invalido');
    yyerrok;
    $$ = { tipo: 'formulario', estilos: [], elementos: [], submit:
null };
}
;

```

```

submit_opcional
: /* vacío */
  { $$ = null; }
| ENVIAR bloc_submit
  { $$ = $2; }
;

lista_inputs
: /* vacío */
  { $$ = []; }
| lista_inputs input_elemento
  { $$ = $1.concat([$2]); }
| lista_inputs error {
  registrarErrorSintacticoActual('Input invalido dentro de
formulario');
  yyerrok;
  $$ = $1;
}
;

input_elemento
: INPUT_TEXTO lista_propiedades_input
  { $$ = { tipo: 'input_text', config: $2 }; }
| INPUT_NUMERO lista_propiedades_input
  { $$ = { tipo: 'input_number', config: $2 }; }
| INPUT_BOOLEANO lista_propiedades_input
  { $$ = { tipo: 'input_bool', config: $2 }; }
;

lista_propiedades_input
: MENOR lista_estilos MAYOR PARENTESIS_ABRE lista_entrada
PARENTESIS_CIERRA
  { $$ = { estilos: $2, propiedades: $5 }; }
| PARENTESIS_ABRE lista_entrada PARENTESIS_CIERRA
  { $$ = { estilos: [], propiedades: $2 }; }
;

lista_entrada
: entrada
  { $$ = [$1]; }
| lista_entrada COMA entrada
  { $$ = $1.concat([$3]); }
;

```

entrada

```
: ID_FORMULARIO DOS_PUNTOS CADENA
  { $$ = { propiedad: 'id', valor: $3 }; }
| ETIQUETA_FORMULARIO DOS_PUNTOS CADENA
  { $$ = { propiedad: 'label', valor: $3 }; }
| VALOR_FORMULARIO DOS_PUNTOS valor_entrada
  { $$ = { propiedad: 'value', valor: $3 }; }
;
```

valor_entrada

```
: NUMERO
  { $$ = Number($1); }
| CADENA
  { $$ = $1; }
| VERDADERO
  { $$ = true; }
| FALSO
  { $$ = false; }
| VARIABLE
  { $$ = { tipo: 'variable', valor: $1 }; }
;
```

bloc_submit

```
: MENOR lista_estilos MAYOR LLAVE_ABRE lista_propiedades_submit
LLAVE_CIERRA
  { $$ = { tipo: 'submit', estilos: $2, propiedades: $5 }; }
| LLAVE_ABRE lista_propiedades_submit LLAVE_CIERRA
  { $$ = { tipo: 'submit', estilos: [], propiedades: $2 }; }
;
```

lista_propiedades_submit

```
: propiedad_submit
  { $$ = [$1]; }
| lista_propiedades_submit propiedad_submit
  { $$ = $1.concat([$2]); }
;
```

propiedad_submit

```
: ETIQUETA_FORMULARIO DOS_PUNTOS CADENA
  { $$ = { propiedad: 'label', valor: $3 }; }
| FUNCION DOS_PUNTOS VARIABLE PARENTESIS_ABRE lista_argumentos
PARENTESIS_CIERRA
```

```

    { $$ = { propiedad: 'function', funcion: $3, argumentos: $5 }; }
;

lista_argumentos
: argumento
  { $$ = [$1]; }
| lista_argumentos COMA argumento
  { $$ = $1.concat([$3]); }
;

argumento
: VARIABLE
  { $$ = { tipo: 'variable', valor: $1 }; }
| REFERENCIA_CAMPO
  { $$ = { tipo: 'referencia_form', valor: $1 }; }
;

logica
: ciclo_for
  { $$ = $1; }
| ciclo_for_each
  { $$ = $1; }
| condicional
  { $$ = $1; }
| switch_stmt
  { $$ = $1; }
| error LLAVE_CIERRA {
    registrarErrorSintacticoActual('Bloque logico invalido');
    yyerrok;
    $$ = null;
  }
;

ciclo_for
: PARA CADA PARENTESIS_ABRE VARIABLE DOS_PUNTOS VARIABLE
PARENTESIS_CIERRA LLAVE_ABRE lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'for_each_simple', origen: $4, item: $6, cuerpo: $9,
vacio: null }; }
| PARA CADA PARENTESIS_ABRE VARIABLE DOS_PUNTOS VARIABLE
PARENTESIS_CIERRA LLAVE_ABRE lista_elementos LLAVE_CIERRA VACIO
LLAVE_ABRE lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'for_each_simple', origen: $4, item: $6, cuerpo: $9,
vacio: $13 }; }

```

```

;

ciclo_for_each
: PARA PARENTESIS_ABRE lista_for_vars PARENTESIS_CIERRA SEGUIMIENTO
VARIABLE LLAVE_ABRE lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'for_each_track', pares: $3, indice: $6, cuerpo: $8,
vacio: null }; }
  | PARA PARENTESIS_ABRE lista_for_vars PARENTESIS_CIERRA SEGUIMIENTO
VARIABLE LLAVE_ABRE lista_elementos LLAVE_CIERRA VACIO LLAVE_ABRE
lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'for_each_track', pares: $3, indice: $6, cuerpo: $8,
vacio: $12 }; }
;

lista_for_vars
: for_var
  { $$ = [$1]; }
  | lista_for_vars COMA for_var
  { $$ = $1.concat([$3]); }
;

for_var
: VARIABLE DOS_PUNTOS VARIABLE
  { $$ = { origen: $1, actual: $3 }; }
;

condicional
: SI PARENTESIS_ABRE expresion PARENTESIS_CIERRA LLAVE_ABRE
lista_elementos LLAVE_CIERRA
  { $$ = { tipo: 'if', condicion: $3, cuerpo: $6, elseIfs: [], else:
null }; }
  | SI PARENTESIS_ABRE expresion PARENTESIS_CIERRA LLAVE_ABRE
lista_elementos LLAVE_CIERRA lista_else
  {
    const partesElseIf = $8.filter((x) => x.tipo === 'else_if');
    const parteElse = $8.find((x) => x.tipo === 'else') || null;
    $$ = { tipo: 'if', condicion: $3, cuerpo: $6, elseIfs:
partesElseIf, else: parteElse };
  }
;

lista_else
: else_parte

```

```

    { $$ = [$1]; }
| lista_else else_parte
    { $$ = $1.concat([$2]); }
;

else_parte
: SINO PARENTESIS_ABRE expresion PARENTESIS_CIERRA LLAVE_ABRE
lista_elementos LLAVE_CIERRA
    { $$ = { tipo: 'else_if', condicion: $3, cuerpo: $6 }; }
| SINO LLAVE_ABRE lista_elementos LLAVE_CIERRA
    { $$ = { tipo: 'else', cuerpo: $3 }; }
;

switch_stmt
: SWITCH PARENTESIS_ABRE VARIABLE PARENTESIS_CIERRA LLAVE_ABRE
lista_casos LLAVE_CIERRA
    { $$ = { tipo: 'switch', variable: $3, casos: $6, defecto: null }; }
| SWITCH PARENTESIS_ABRE VARIABLE PARENTESIS_CIERRA LLAVE_ABRE
lista_casos caso_defecto LLAVE_CIERRA
    { $$ = { tipo: 'switch', variable: $3, casos: $6, defecto: $7 }; }
;

lista_casos
: caso
    { $$ = [$1]; }
| lista_casos COMA caso
    { $$ = $1.concat([$3]); }
;

caso
: CASO CADENA LLAVE_ABRE lista_elementos LLAVE_CIERRA
    { $$ = { valor: $2, cuerpo: $4 }; }
;

caso_defecto
: DEFECTO LLAVE_ABRE lista_elementos LLAVE_CIERRA
    { $$ = { cuerpo: $3 }; }
;

expresion
: comparacion
    { $$ = $1; }
| expresion AND comparacion

```

```
    { $$ = { op: '&&', left: $1, right: $3 }; }  
  | expresion OR comparacion  
    { $$ = { op: '||', left: $1, right: $3 }; }  
  ;
```

comparacion

```
: termino  
  { $$ = $1; }  
| termino IGUALDAD termino  
  { $$ = { op: '==', left: $1, right: $3 }; }  
| termino NO_IGUAL termino  
  { $$ = { op: '!=', left: $1, right: $3 }; }  
| termino MENOR termino  
  { $$ = { op: '<', left: $1, right: $3 }; }  
| termino MAYOR termino  
  { $$ = { op: '>', left: $1, right: $3 }; }  
;
```

termino

```
: factor  
  { $$ = $1; }  
| termino SUMA factor  
  { $$ = { op: '+', left: $1, right: $3 }; }  
| termino GUION factor  
  { $$ = { op: '-', left: $1, right: $3 }; }  
| termino MULTIPLICADOR factor  
  { $$ = { op: '*', left: $1, right: $3 }; }  
| termino DIVISION factor  
  { $$ = { op: '/', left: $1, right: $3 }; }  
;
```

factor

```
: NUMERO  
  { $$ = Number($1); }  
| CADENA  
  { $$ = $1; }  
| VARIABLE  
  { $$ = { tipo: 'variable', valor: $1 }; }  
| VERDADERO  
  { $$ = true; }  
| FALSO  
  { $$ = false; }  
| PARENTESIS_ABRE expresion PARENTESIS_CIERRA
```



```
{ $$ = $2; }  
| VARIABLE CORCHETE_ABRE NUMERO CORCHETE_CIERRA  
  { $$ = { tipo: 'acceso_arreglo', variable: $1, indice: Number($3) };  
}  
;
```

Diagrama de Clases

Diagrama de clases del Back-end

Para poder visualizar la forma en que el backend se ha estructurado se deja un enlace al diagrama de clases que la componen.

<https://drive.google.com/file/d/1-oJTnq-4cmYI6hqOUmMU-bg1p8Ael6t9/view?usp=sharing>